

Software Architecture and Design Specification

Project: PESU Event Management System (PESU-EMS)

Version: 1.0

Authors: Team 7 (Shayna Zaidi Gupta, Shaurya Ishan, Shriya C Rao, Sham Santosh.S)

Date: 03-10-2026

Revision History

version	Date	Author	Change summary
1.0	3-10-2026	Team 7	Initial draft of architecture and design, based on SRS

Approvals

Role	Name	Signature/Date
QA Lead	Shayna Zaidi Gupta	3.10.2026
Test engineer	Shriya C Rao	3.10.2026
Dev Lead	Shaurya Ishan	3.10.2026
Product Owner	Sham Santosh	3.10.2026

1. Introduction

1.1 Purpose

This document explains how PESU-EMS is built and how its parts fit together. PESU-EMS is a website where organizers post university events and students find them and sign up.

1.2 Scope

Scope includes the main features mentioned in the SRS. Users can log in either as students or organizers. Organizers can create, update, and cancel events, view registered students, record attendance etc. Students can browse and search for events, and register for available events

before the deadline or seats run out. Features such as online payments and automatic certificate generation are not in scope for now but are planned for future improvements.

1.3 Audience

Developers who build the system, testers, faculty who review it, organizers who use it, and administrators who run it.

1.4 Definitions

PESU-EMS stands for PES University Event Management System. SRS is the requirements document, STP is the test plan, and RTM is the requirements traceability matrix. REST is the API style used in the system. RBAC controls what each user role can access. STRIDE is used to identify security threats such as spoofing, tampering, information disclosure, denial of service, and privilege escalation.

2. Document Overview

2.1 How to use this document

This document contains the design work for PESU-EMS: a component diagram, a description of each component, the architecture pattern, the technology used, risks, how requirements map to components, security, two sequence diagrams, API details, error handling, and UX.

2.2 Related Documents

Software Requirements Specification for PESU-EMS, RTM, Software Engineering Project Proposal, Software Test Plan.

3. Architecture

3.1 Goals & Constraints

Goals

- Keep event posting, browsing, and registration in one system.
- Handle user roles and event capacity on the server.
- Keep registration and attendance data accurate and consistent.
- Make sure normal requests are handled within about 2 seconds.
- Divide the system into modules so the team can work on different parts at the same time.

Constraints

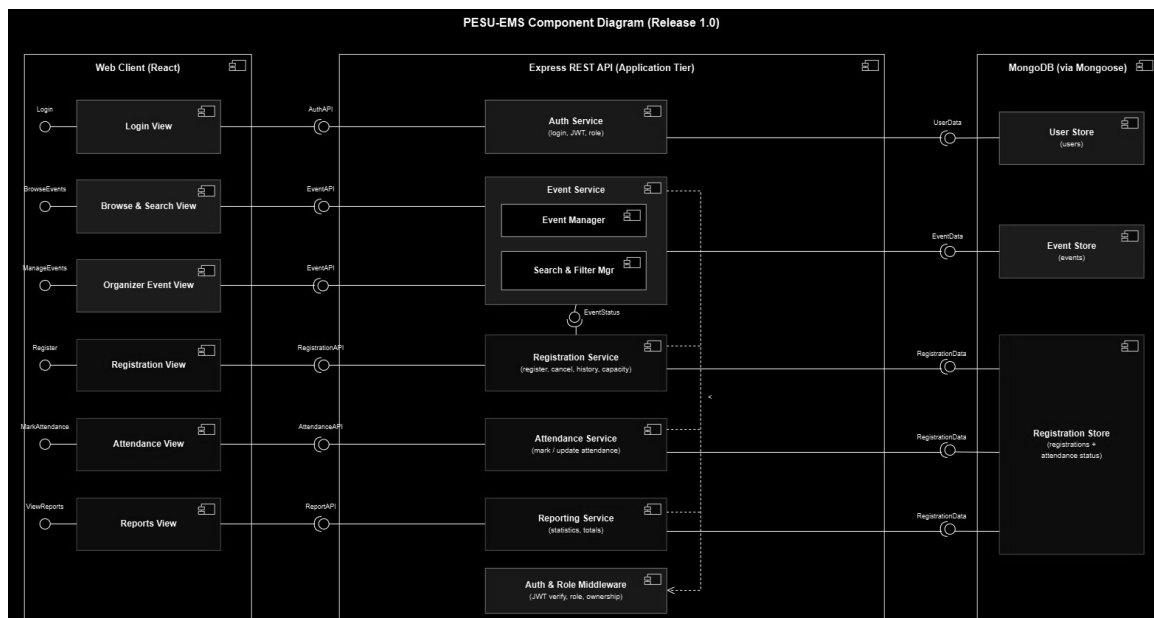
- Access is based on user roles, and login checks the college email format.
- Students cannot access organizer features, and organizers can modify only their own events.

- Passwords must be stored securely, and HTTPS is required in production.
- Event capacity must be enforced by the backend.
- The project must be completed within the timeline.

3.2 Stakeholders & Concerns

1. Students: want to find events easily, register quickly, and see their past registrations.
2. Event Organizers: want to create events, see who registered, and mark attendance easily.
3. Course Faculty: Ensure the design is complete and properly matches the SRS requirements.
4. Developers : want each module to be built and tested separately.

3.3 Component (UML) Diagram



3.4 Component Descriptions

Stakeholder	Main Concern
Students	Find events easily, register quickly, and view their registration history.

Organizers	Manage event capacity, participant lists, attendance, and statistics correctly, while ensuring only they can modify their events.
Course Faculty	Ensure the design is complete and properly matches the SRS requirements.
Developers	Have clear module boundaries so the team can work on different parts independently.
Test Engineers	Have clear requirements and interfaces that can be tested separately.
System Administrators	Easy deployment, secure HTTPS, and keeping sensitive information out of the source code.

3.5 Chosen Architecture Pattern and Rationale

We chose a layered architecture with three tiers. React forms the presentation layer, while the Express API handles the main application logic through four modules: Auth, Events, Registration, and Attendance & Reports. The Data Access Layer and database are placed below these modules. Each request passes through authentication, role checking, the required module, and then the Data Access Layer. It matches how our team split the work, lets each module be tested on its own, and is simple to build in the given timeline.

We rejected microservices. Separate services would add deployment and networking work that a project this size does not need. The cost of our choice is a single deployable, so a bug in one module can take the others down.

3.6 Technology Stack & Data Stores

Client: React is used for the frontend.

Server: Node.js with Express is used to build the REST API, with JSON over HTTPS.

Login: JWT tokens given after a successful sign-in. Passwords are hashed

Data Store: MongoDB

Diagram Tool: StarUML is used to create the system diagrams.

3.7 Risks & Mitigations

Risk	Mitigation
Too many students register at the same time and go over the event capacity	Check capacity and save the registration in one single database step
A problem in the single deployment could affect all modules.	Clear module boundaries are maintained so the API can be separated later if needed.
Delaying the login module could block development of the other modules.	Login will be developed and tested early and will be shared with the team regularly.

3.8 Traceability to Requirements

The table shows how each group of SRS requirements is connected to the relevant module and design section.

Req. ID	Requirement
AUTH-01-07	Auth Service
EVENT-01-08	Event Service
DISC-01-06	Event Service (discovery/search)
REG-01-08	Registration Service
HIST-01-04	Registration Service (history)
PART-01-04	Registration Service (participants)
ATT-01-06	Attendance & Reporting Service
REP-01-05	Attendance & Reporting Service (reports)

3.9 Security Architecture

Security risks (STRIDE method) and how we handle them:

- Spoofing : users sign in with college credentials and get a signed token, which is needed for every protected request.
- Tampering: the server checks every token. Capacity and attendance updates are checked on the server, not trusted from the browser.
- Repudiation: each registration and attendance record stores the user ID and time.
- Information disclosure: all traffic uses HTTPS. Passwords are hashed and never logged. A student's history is visible only to that student and the organizers of their events.

- Denial of service :all inputs are checked
- Elevation of privilege: the server checks the user's role before allowing organizer-only actions like creating events, marking attendance, or viewing reports.

4. Design

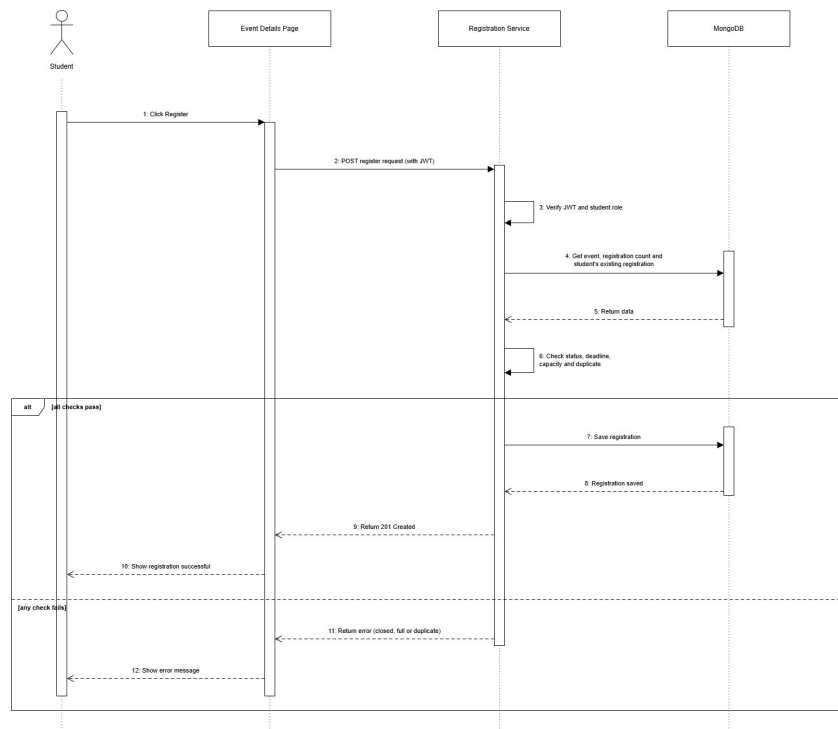
4.1 Design Overview

PESU-EMS has four separate modules: Authentication, Event Management & Discovery, Registration & Participation, and Attendance & Reporting. Each module has its own database collection and is built and tested by one team member. The React client uses all four through the same REST API.

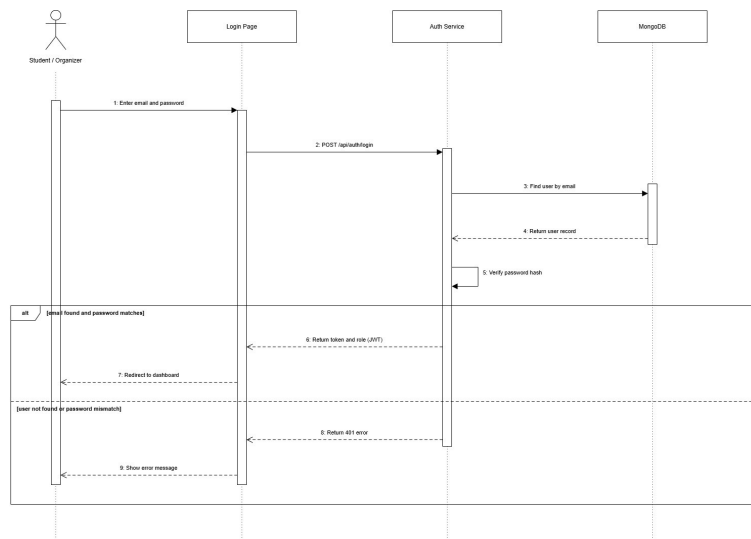
4.2 UML Sequence Diagrams

At least 2 sequence diagrams covering 2 flows specific to your project

Student registering for event:



Login:



4.3 API Design

Interface definition for 2 components.

1)Registration service

Endpoint: /api/registrations

Method: POST

Request: { studentId, eventId }

Response: { status, registration, updatedCapacity }

Errors: 403 Not a student, 404 Event not found, 409 Event full or deadline passed.

2)Attendance service

Endpoint: /api/attendance/:registrationId

Method: PATCH

Request: { status: "Present" | "Absent" }

Response: { status, registration }

Errors: 403 Not the organizer of this event, 404 Registration not found.

4.4 Error Handling, Logging & Monitoring

Standardized error messages. No sensitive info in logs. Monitoring includes error rate, response time, and failed registration.

4.5 UX Design

The website has the same menu on every page and separate views for students and organizers. Clear messages appear for common problems, such as wrong password, event full, or deadline passed. The main actions take only a few clicks.

4.6 Open Issues & Next Steps

Future enhancements: Waitlist for full events, email notifications for registration and cancellation, payments for paid events.

5. Appendices

5.1 Glossary: ATM, PIN, PCI-DSS, TLS, ADR.

5.2 References: PESU-EMS Software Requirements Specification (SRS), PESU-EMS Software Test Plan (STP)

5.3 Tools: Star UML, Github